UEPB
Universidade Estadual da Paraíba
Departamento de Computação – DC
Engenharia de Software II
Professor: Dra. Sabrina de Figueirêdo Souto

ASCENDINO MARTINS DE AZEVEDO NETO

ERIC NATHAN ALMEIDA DO NASCIMENTO

VITOR JESUS MAMEDE SOARES

WERLYS KAYAN SOARES DOS SANTOS

RELATÓRIO - RELEASE 03

CAMPINA GRANDE – PB

19 de março de 2026

SUMÁRIO

1. INTRODUÇÃO 3
2. DESCRIÇÃO DA ARQUITETURA 3
3. FUNCIONALIDADES DESENVOLVIDAS 6
3.1 Sistema de Pedidos e Restaurantes 6
3.2 Gerenciamento de Entregadores 7
3.3 Gerenciamento de Restaurantes 7
3.4 Sistema de Avaliações 8
3.5 Sistema de Notificações 9
4. TESTES E COBERTURA DE CÓDIGO 9
4. CONCLUSÃO 11
5. REFERÊNCIAS 12

1. INTRODUÇÃO

A Release 3 do sistema UberPB representa a release surpresa que estava prevista desde o início do projeto de evolução do projeto. Nesta fase, o foco principal foi a inclusão do Uber Eats no projeto UberPB já criado anteriormente, aperfeiçoamento da experiência de uso e a ampliação dos recursos de pagamento, avaliação e histórico de corridas, de modo que o usuário possa, além de solicitar corridas, solicitar deliveries de restaurantes, buscando aproximar o sistema de um ambiente mais realista e completo. Além disso, por proposta da professora houve uma mudança nos integrantes da equipe, em que, em relação às releases passadas permaneceu apenas o integrante Ascendino Martins e foram adicionados ao grupo os integrantes Eric Nathan, Vitor Jesus e Werlys Santos.

Com base nas arquiteturas estabelecida na Release 1 e na Release 2, a equipe deu continuidade à implementação de novos requisitos e adicionou novas funcionalidades para alguns requisitos já existentes, distribuídos em duas sprints (Sprint 5 e Sprint 6). Esta versão introduz novos papéis no sistema, como restaurante e entregador, além de implementar um fluxo completo de pedidos, desde a seleção de itens até a entrega final ao cliente. Foram desenvolvidos mecanismos de validação de documentos, gerenciamento de pedidos imediatos e agendados, sistema de avaliações e notificações em tempo real.

Dessa forma, a Release 03 amplia a complexidade e a robustez do sistema, aproximando-o de aplicações reais utilizadas no mercado, ao mesmo tempo em que mantém a organização arquitetural e os princípios de modularidade e escalabilidade.

2. DESCRIÇÃO DA ARQUITETURA

O sistema UberPB mantém sua arquitetura em camadas, composta por modelos (model), serviços (service), repositórios (repo) e interface de usuário (cli). Essa estrutura foi expandida para suportar o novo módulo de entrega de alimentos.

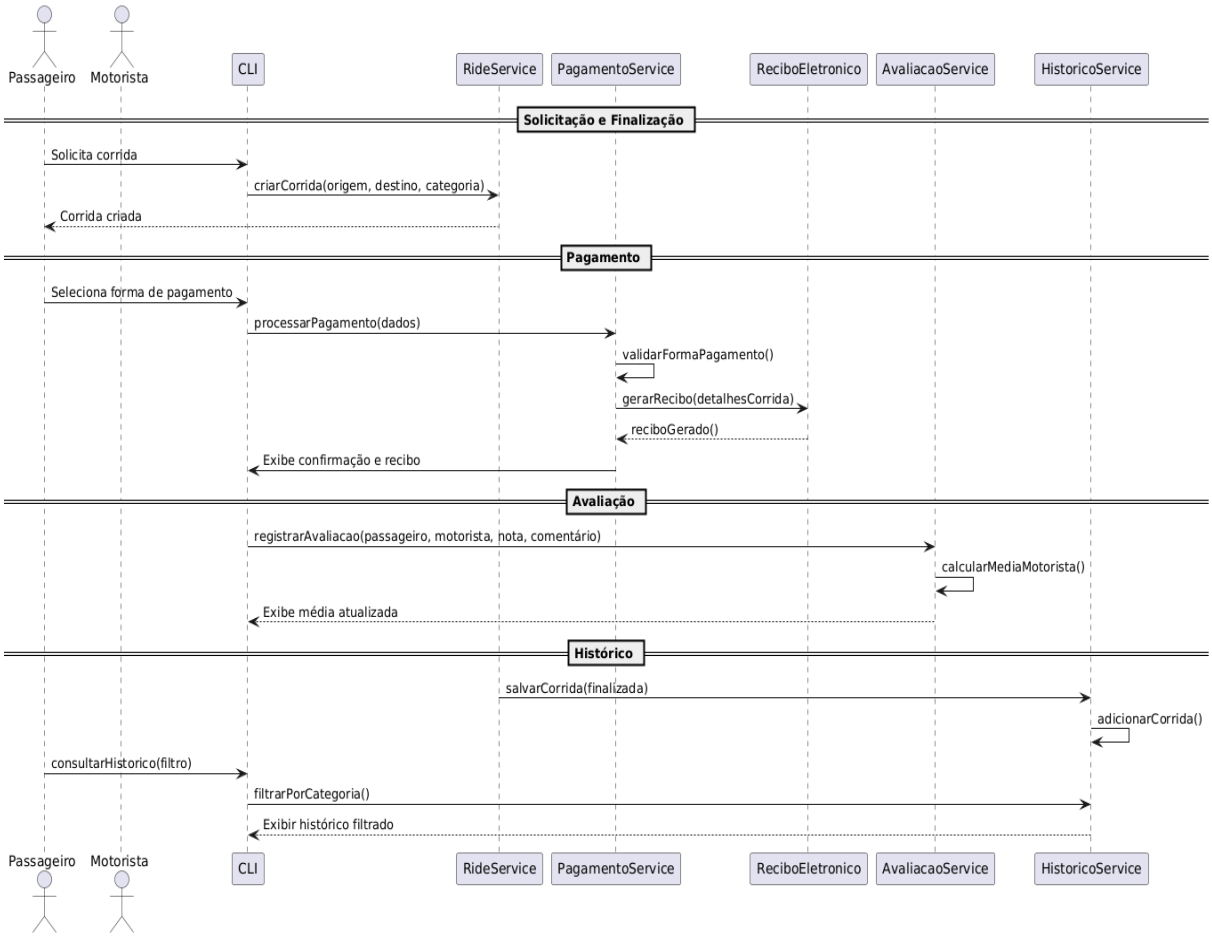
Na camada de model, foram adicionadas novas entidades fundamentais, como Restaurant, MenuItem, Order e Delivery, além de enums responsáveis pelo controle de estados, como OrderStatus, OrderType e DeliveryStatus. Essas classes representam os principais elementos do domínio do sistema de entregas.

Na camada de service, novos serviços foram implementados para coordenar as regras de negócio:

- **RestaurantService:** gerenciamento de restaurantes e cardápios;
- **OrderService:** criação, confirmação e rastreamento de pedidos;
- **DeliveryService:** cadastro e validação de entregadores;
- **DeliveryAssignmentService:** responsável pela atribuição automática de entregadores com base na proximidade;
- **AvaliacaoService:** gerenciamento das avaliações.
- **NotificationService:** envio de notificações em tempo real.

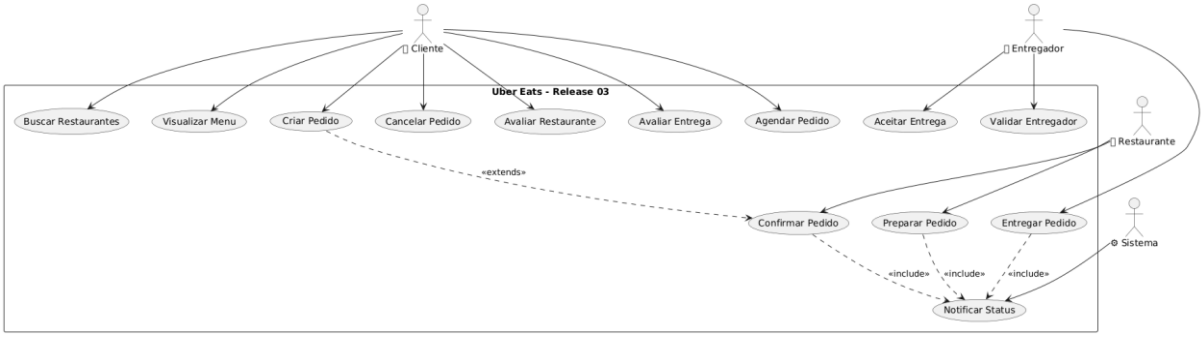
A camada de persistência (repo) foi estendida para armazenar dados relacionados a pedidos, restaurantes, entregadores e avaliações. Já a camada de interface (cli) passou a incluir novos menus interativos, permitindo o gerenciamento completo do fluxo de pedidos por diferentes tipos de usuários.Essa organização mantém a separação de responsabilidades e favorece a manutenção e evolução do sistema.

Figura 01: Diagrama de Sequência.



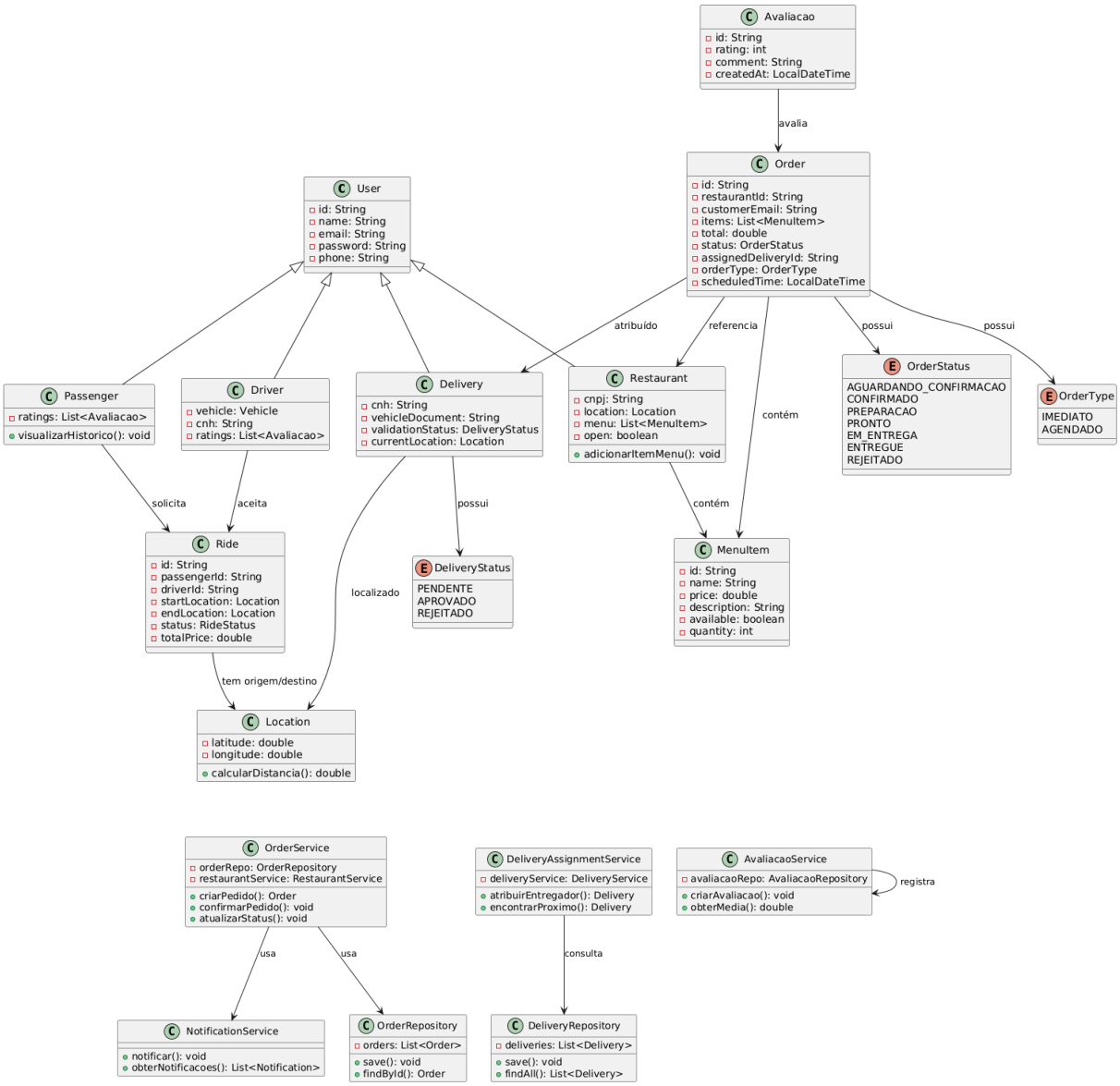
Fonte: Autoria Própria, 2026.

Figura 02: Diagrama de Caso de Uso.



Fonte: Autoria Própria, 2026.

Figura 03: Diagrama de Classes.



Fonte: Autoria Própria, 2026.

3. FUNCIONALIDADES DESENVOLVIDAS

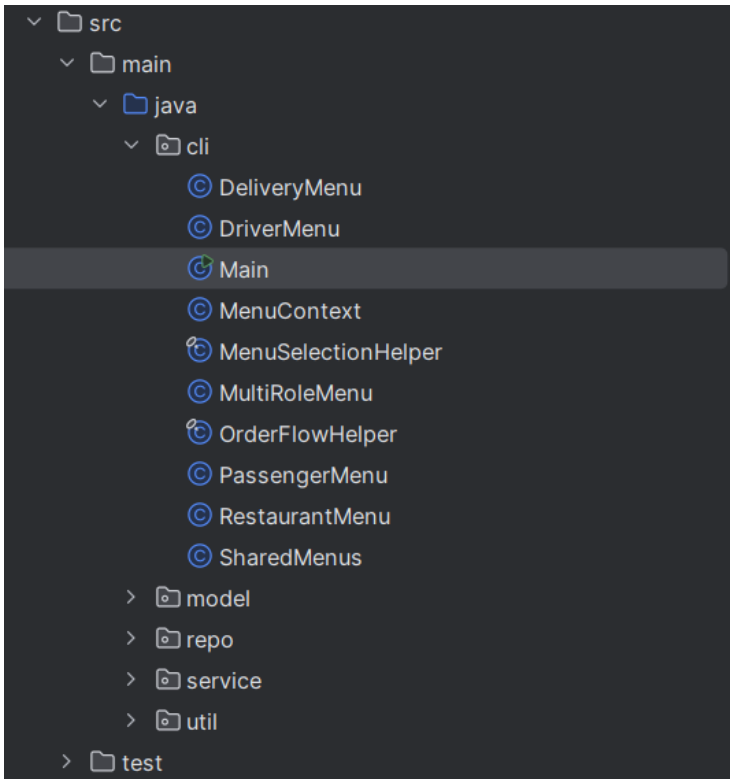
3.1 Sistema de Pedidos e Restaurantes

O módulo de pedidos permite que clientes busquem restaurantes, visualizem cardápios e realizem pedidos de forma simples e intuitiva. Cada pedido pode ser do tipo imediato ou agendado, oferecendo maior flexibilidade ao usuário.

O sistema realiza automaticamente o cálculo do valor total, considerando subtotal dos itens, taxa de entrega e possíveis descontos. Após a criação, o pedido passa por diferentes estados, como aguardando confirmação, preparação, entrega e finalização. Essa funcionalidade representa o núcleo da integração do Uber Eats ao sistema UberPB.

Além disso, o Main passou por uma refatoração em relação às releases passadas, em que todas as opções ficavam no menu principal, fazendo com que ele fosse muito grande. Dessa forma, o CLI que antes pussuía apenas a Classe Main, foi dividido em categorias.

Figura 04: Solicitando a corrida mostrando o preço dinâmico.



Fonte: Eclipse IDE.

3.2 Gerenciamento de Entregadores

Foi implementado um sistema completo de cadastro e validação de entregadores, incluindo verificação de CNH e documento do veículo. Apenas entregadores validados podem participar do processo de entrega.

O sistema também realiza a atribuição automática de entregadores com base na proximidade, utilizando cálculos de distância entre o restaurante e os entregadores disponíveis. Essa abordagem otimiza o tempo de entrega e melhora a eficiência do sistema.

Figura 05: Simulação de Cadastro de Entregador.

```
=== Menu Principal ===
1 - Cadastrar Passageiro
2 - Cadastrar Motorista
3 - Cadastrar Entregador
4 - Cadastrar Restaurante
5 - Fazer Login
6 - Listar Usuários (Admin)
7 - Listar Categorias de Veículos
8 - Ajustar Tarifa Dinâmica (Admin)
0 - Sair
> 3
=== Cadastro de Entregador ===
Nome: Ascendino Martins de Azevedo Neto
Email: ascendinoM@gmail.com
CPF: 12362110494
Telefone: 83999450523
CNH (11 dígitos): 34567891095
Placa do veículo: ASC13E4

Entregador cadastrado com sucesso!
Nome: Ascendino Martins de Azevedo Neto
Email: ascendinoM@gmail.com

Deseja fazer login agora? (s/n):
```

Fonte: Eclipse IDE.

3.3 Gerenciamento de Restaurantes

O cadastro de Restaurantes também foi implementado com sistema de cadastro e validação de entregadores, incluindo verificação de CNPJ e endereço. O responsável pelo restaurante pode criar o seu cardápio, incluindo o item, a descrição e o seu preço.

No sistema o Restaurante também pode ver os seus pedidos, notificações e alternar o status e o funcionamento do estabelecimento.

Figura 06: Simulação de Cadastro de Restaurante.

```
=== Menu Principal ===
1 - Cadastrar Passageiro
2 - Cadastrar Motorista
3 - Cadastrar Entregador
4 - Cadastrar Restaurante
5 - Fazer Login
6 - Listar Usuários (Admin)
7 - Listar Categorias de Veículos
8 - Ajustar Tarifa Dinâmica (Admin)
0 - Sair
> 4
=== Cadastro de Restaurante ===
Nome: Restaurante do Vitor
Email: vitorrestaurante@gmail.com
CNPJ (14 dígitos): 12345678901234
Endereço: Avenida Juvencio Arruda
Latitude (opcional, 0 se desconhecido): 0
Longitude (opcional, 0 se desconhecido): 0

Restaurante cadastrado com sucesso!
Nome: Restaurante do Vitor
Status: Ativo
```

Fonte: Eclipse IDE.

3.4 Sistema de Avaliações

A Release 03 introduz um sistema de avaliações para restaurantes e entregadores. Após a conclusão de um pedido, o cliente pode avaliar tanto o estabelecimento quanto o serviço de entrega. Além disso, o Entregador pode avaliar tanto o Cliente como o Restaurante, e o Restaurante, por sua vez, pode avaliar o entregador.

As avaliações são armazenadas e utilizadas para cálculo de médias, contribuindo para a reputação dos participantes e incentivando a qualidade do serviço.

Figura 07: Menu do Entregador que possui a opção de Avaliar Cliente/Restaurante.

```
Deseja fazer login agora? (s/n): s

=== Menu Entregador - Ascendino Martins de Azevedo Neto ===
Avaliação média: 0,00 (0 avaliações)
Status: Ativo
Localização: Localização inicial #8caccdab
1 - Atualizar Localização
2 - Ver Notificações
3 - Alternar Disponibilidade
4 - Ver Minhas Entregas
5 - Avaliar Cliente/Restaurante
0 - Sair
>
```

Fonte: Eclipse IDE.

3.5 Sistema de Notificações

Um sistema de notificações em tempo real foi implementado para informar usuários sobre mudanças de status dos pedidos. Notificações são enviadas durante todo o ciclo, incluindo confirmação, início de preparo, saída para entrega e finalização.

Esse mecanismo melhora a experiência do usuário, fornecendo transparência e acompanhamento contínuo do pedido.

Figura 08: Menu do Restaurante que possui a opção de ver notificações.

```
=== Menu Restaurante - Restaurante do Vitor  ===
Avaliação média: 0,00 (0 avaliações)
Status: Ativo | Aberto: Sim
1 - Ver Pedidos Pendentes
2 - Atualizar Status do Pedido
3 - Ver Notificações
4 - Gerenciar Cardápio
5 - Alternar Status (Ativo/Inativo)
6 - Alternar Funcionamento (Aberto/Fechado)
7 - Avaliar Cliente/Entregador
0 - Sair
>
```

Fonte: Eclipse IDE.

4. TESTES E COBERTURA DE CÓDIGO

A Release 03 manteve o compromisso com a qualidade do software, alcançando aproximadamente 82% de cobertura de testes utilizando o framework JUnit e o plugin Emma.

Foram implementados testes para os principais serviços do sistema, incluindo:

- **OrderServiceTest** (criação e gerenciamento de pedidos);
- **DeliveryServiceTest** (cadastro e validação de entregadores);
- **RestaurantServiceTest** (gerenciamento de restaurantes);
- **DeliveryAssignmentServiceTest** (atribuição de entregadores);
- **AvaliacaoServiceTest** (avaliações e médias).

Esses testes garantem a confiabilidade do sistema e reduzem a ocorrência de falhas em funcionalidades críticas.

Figura 09: Trecho de código do Teste OrderServiceTest.

```
        restaurant = restaurantService.register(
                name: "Pizza Top",
                email: "pizza@email.com",
                cnpj: "12345678901234",
                new Location( address: "Rua A")
        );

        restaurant.addMenuItem(new MenuItem( name: "Pizza Calabresa",  description: "Tradicional",  price: 40));
        restaurant.addMenuItem(new MenuItem( name: "Refrigerante",  description: "Lata",  price: 10));
    }

    @Test   WerlysKayan
```

Fonte: Eclipse IDE.

Figura 10: Trecho de código do Teste DeliveryServiceTest.

```
    @Test   Ascendino-Neto +1
    void shouldApproveDeliveryWithValidDocuments() {
        Delivery delivery = service.register(
                name: "Maria",
                email: "maria@email.com",
                document: "12345678901",
                phone: "11999999999",
                cnh: "12345678901",
                vehicleDocument: "CRLV123");

        assertEquals(DeliveryStatus.APROVADO, delivery.getValidationStatus());
    }
```

Fonte: Eclipse IDE.

Figura 11: Trecho de código do Teste RestaurantServiceTest.

```
@Test   Erick Nathan +1
void shouldRegisterRestaurant() {

    Restaurant restaurant = service.register(
            name: "Pizza Top",
            email: "pizza@email.com",
            cnpj: "12345678901234",
            new Location( address: "Rua A")
    );

    assertNotNull(restaurant.getId());
    assertEquals( expected: "Pizza Top", restaurant.getName());
}
```

Fonte: Eclipse IDE

Figura 12: Trecho de código do Teste DeliveryAssignmentServiceTest.

```
@Test  vitorjsss
void shouldFindNearestAvailableDelivery() {
        // Criar entregadores em localizações diferentes
        Delivery delivery1 = deliveryService.register(
                        name: "João",  email: "joao@email.com",  document: "12345678901",
                        phone: "11999999999",  cnh: "12345678901",  vehicleDocument: "CRLV123");
        delivery1.setCurrentLocation(new Location( address: "Local 1",  description: "",  latitude: 0,  longitude: 0));

        Delivery delivery2 = deliveryService.register(
                        name: "Maria",  email: "maria@email.com",  document: "12345678902",
                        phone: "11999999998",  cnh: "12345678902",  vehicleDocument: "CRLV456");
        delivery2.setCurrentLocation(new Location( address: "Local 2",  description: "",  latitude: 10,  longitude: 10));

        Location restaurantLocation = new Location( address: "Restaurante",  description: "",  latitude: 1,  longitude: 1);

        // Encontrar o mais próximo
        Delivery nearest = assignmentService.findNearestAvailableDelivery(restaurantLocation);

        assertNotNull(nearest);
        assertEquals( expected: "João", nearest.getName());
}
```

Fonte: Eclipse IDE.

Figura 13: Trecho de código do Teste AvaliacaoServiceTest.

```
@Test  vitorjsss
public void restaurantCanRateCustomerAndDelivery() {
    avaliacaoService.restaurantRatesCustomer(deliveredOrder.getId(),  restaurantId: "rest-1",  nota: 4,  comentario: "Tudo certo");
    avaliacaoService.restaurantRatesDelivery(deliveredOrder.getId(),  restaurantId: "rest-1",  nota: 2,  comentario: "Atrasou");

    assertEquals( expected: 4.0, avaliacaoService.getAverageRatingForCustomer( customerEmail: "cliente@teste.com"),  delta: 0.001);
    assertEquals( expected: 2.0, avaliacaoService.getAverageRatingForDelivery( deliveryId: "del-1"),  delta: 0.001);
}
```

Fonte: Eclipse IDE

4. CONCLUSÃO

A Release 03 do sistema UberPB representou uma etapa de expansão significativa do projeto, incorporando os requisitos relacionados ao módulo de entrega de alimentos (Uber Eats), conforme definido no escopo do projeto. Diferentemente das releases anteriores, que estavam focadas principalmente no serviço de transporte, esta fase introduziu novos fluxos de negócio, ampliando o sistema para um ambiente multifuncional.

Durante o desenvolvimento, foram implementados os requisitos destacados para esta etapa (RF1, RF2, RF19 a RF28 e RF14), contemplando funcionalidades como cadastro e autenticação de novos perfis (restaurantes e entregadores), visualização de cardápios, criação e

acompanhamento de pedidos, atribuição automática de entregadores, além de avaliações e notificações em tempo real. Essas funcionalidades foram distribuídas entre as Sprints 5 e 6, seguindo o planejamento estabelecido, o que permitiu uma evolução incremental e organizada do sistema.

A arquitetura em camadas foi mantida e estendida para suportar os novos componentes, garantindo a separação de responsabilidades e facilitando a integração entre os módulos de transporte e entrega. Além disso, foram aplicados testes unitários com cobertura superior a 80%, assegurando a confiabilidade das principais funcionalidades implementadas.

A equipe manteve o uso de boas práticas de engenharia de software, incluindo padrões de projeto, persistência local de dados e tratamento de erros, conforme exigido nas diretrizes do projeto. Como resultado, o sistema evoluiu para uma solução mais robusta, capaz de simular com maior fidelidade aplicações reais de mobilidade e delivery.

Dessa forma, a Release 03 consolida o UberPB como uma plataforma integrada, preparada para futuras expansões.

5. REFERÊNCIAS

PLANTTEXT. PlantText: PlantUML Editor Online – Free & Fast UML Diagram Tool. Disponível em: https://www.planttext.com/ Acesso em: 24 out. 2025.

WINTERS, Titus; MANSHRECK, Tom; WRIGHT, Hyrum. Software engineering at Google: lessons learned from programming over time. 1. ed. Sebastopol: O’Reilly Media, 2020.